



Hochiminh City University of Technology
Computer Science and Engineering
[CO1027] - Fundamentals of C++ Programming

Template

Lecturer: Duc Dung Nguyen
Credits: 3

Outcomes

- ❖ Understand the concept of template in OOP.

Outline

- ❖ Introduction
- ❖ Function Templates
- ❖ Class Templates
- ❖ Class Templates and Non-type Parameters
- ❖ Templates and Inheritance
- ❖ Templates and friends
- ❖ Templates and static Members

Introduction

- ❖ Templates
 - ❖ Easily create a large range of related functions or classes
 - ❖ Function template - the blueprint of the related functions
 - ❖ Template function - a specific function made from a function template

Function Templates

❖ Using function overloading:

```
int addTwoNumbers(int, int);  
float addTwoNumbers(float, float);  
double addTwoNumbers(double, double);
```

❖ Disadvantages:

Function Templates Syntax

❖ The function declaration:

```
template <typename T> //tell the compiler we are using a template

//T represents the variable type: any fundamental type or user-defined type
T functionName(T parameter1, T parameter2, ...);
```

❖ The function definition:

```
template <typename T>
T functionName(T parameter1, T parameter2, ...)
{
    function statements;
}
```

Class Templates

- ❖ Class templates
 - ❖ Allow type-specific versions of generic classes
- ❖ Format:

```
template <class T>  
  
class ClassName{  
    definition  
}
```

- ❖ To create an object of the class, type

```
ClassName< type > myObject;
```

```
Example: Stack< double > doubleStack;
```

Class Templates and Non-type Parameters

❖ Can use non-type parameters in templates

- Default argument
- Treated as **const**

❖ Example:

```
template< class T, int elements >  
Stack< double, 100 > mostRecentSalesFigures;
```

- Declares object of type `Stack< double, 100>`

- This may appear in the class definition:

```
T stackHolder[ elements ]; //array to hold stack
```

- Creates array at compile time, rather than dynamic allocation at execution time

Templates and Inheritance

- ❖ A class template can be derived from a class-template specialization.
- ❖ A class template can be derived from a non-template class.
- ❖ A class-template specialization can be derived from a class-template specialization.
- ❖ A non-template class can be derived from a class-template specialization.

Templates and Friends

- Friendships allowed between a class template and
 - Global function
 - Member function of another class
 - Entire class
- **friend** functions
 - Inside definition of class template **X**:
 - **friend void f1() ;**
 - **f1()** a **friend** of all template classes
 - **friend void f2(X< T > &) ;**
 - **f2(X< int > &)** is a **friend** of **X< int >** only. The same applies for **float**, **double**, etc.
 - **friend void A::f3() ;**
 - Member function **f3** of class **A** is a **friend** of all template classes

Templates and Friends (cont)

- **friend void C< T >::f4(X< T > &);**
 - **C<float>::f4(X< float> &)** is a **friend** of **class X<float>** only
- **friend** classes
 - **friend class Y;**
 - Every member function of **Y** is a friend with every template class made from **X**
 - **friend class Z<T>;**
 - Class **Z<float>** a **friend** of class **X<float>**, etc.

Templates and static Members

- ❖ Non-template class
 - ❖ **static** data members shared between all objects
 - ❖ **static** data members initialized at global scope
- ❖ Template classes
 - ❖ Each class (**int**, **float**, etc.) has its own copy of **static** data members
 - ❖ **static** data members initialized at global scope
 - ❖ Each template class gets its own copy of **static** member functions